分发信件（letters）

每栋宿舍在总房间号中占据一段连续的位置。记 $s[i]$ 为前 i 栋宿舍的房间总数，令 $s[0]=0$，则第 i 栋的号码范围是

$$s[i-1]+1,\ s[i-1]+2,\ \ldots,\ s[i].$$

因此，对于号码 x，只需找到第一个满足 $s[p]\geq x$ 的宿舍 p。前面的 $s[p-1]$ 个房间不属于这一栋，所以楼内房间号是 $x-s[p-1]$。

信件号码已经按严格递增顺序给出。处理下一封信时，它不会回到此前的宿舍，因此可以保留指针 p，只要 $s[p]<x$ 就让它右移。停止时，前一栋的末尾小于 x，当前栋的末尾不小于 x，恰好找到了这封信的位置。

指针从第一栋开始，整个过程最多右移 $n-1$ 次，不会为每封信重新扫描。预处理和回答询问共用 $O(n+m)$ 时间，空间复杂度为 $O(n)$。房间总数最多为 2×10^{15}，房间数、前缀和及信件号码都用 long long 保存。

【参考代码】

```
#include<bits/stdc++.h>
using namespace std;

const int N = 2e5 + 10;
int n,m;
long long s[N];

int main(){
  freopen("letters.in","r",stdin);
  freopen("letters.out","w",stdout);
  ios::sync_with_stdio(false);
  cin.tie(0);
  cin >> n >> m;
  for(int i = 1; i <= n; i++){
    long long x;
    cin >> x;
    s[i] = s[i - 1] + x;
  }
  int p = 1;
  for(int i = 1; i <= m; i++){
    long long x;
    cin >> x;
    while(s[p] < x) p++;
    cout << p << ' ' << x - s[p - 1] << '\n';
  }
  return 0;
}
```

【部分分】

条件	分值	做法与复杂度
$n,m\leq2000$	测试点 $1\sim6$，共 30 分	对每封信都从第一栋开始，将号码依次减去经过的宿舍房间数；当剩余号码不超过当前宿舍的房间数时，得到宿舍编号与楼内号码。总时间 $O(nm)$，空间 $O(n)$。
所有宿舍的房间数相同	测试点 $7\sim9$，共 15 分	设每栋有 c 个房间，号码 x 对应第 $\lfloor(x-1)/c\rfloor+1$ 栋，楼内号码为 $(x-1)\bmod c+1$。总时间 $O(n+m)$，空间 $O(n)$。

公式中先减 1，是为了让每栋的最后一个号码仍落在当前栋。先判断房间数是否全部相同，否则使用第一种做法，就能覆盖两个互不重叠的点组，保证 45 分。

舞伴选择（dance）

直接枚举选中的两对舞伴，需要检查约 k^2 次。检查的条件却只有两种：是否用了同一个男生，是否用了同一个女生。因此，可以先计算所有选择，再减去这两类不合法的选择。

从 k 对舞伴中任选两对，共有 $k(k-1)/2$ 种方案。

如果男生 u 出现在 c_u 对舞伴中，那么从这 c_u 对里任选两对，都会重复使用他，所以要减去 $c_u(c_u-1)/2$。同理，若女生 v 出现在 d_v 对舞伴中，就要减去 $d_v(d_v-1)/2$。

这里不会重复扣除：如果两对舞伴既用了相同男生，又用了相同女生，那么它们就是完全相同的一对，而题目保证没有重复的舞伴对。所有不合法方案又一定重复使用了男生或女生，因此上述扣除恰好覆盖全部不合法方案。

统计每位男生、女生的出现次数后，答案就是

$$\frac{k(k-1)}{2}-\sum_{u=1}^{a}\frac{c_u(c_u-1)}{2}-\sum_{v=1}^{b}\frac{d_v(d_v-1)}{2}.$$

每组用 $O(a+b+k)$ 时间、$O(a+b+k)$ 空间。总时间为 $O(\sum(a+b+k))$，空间取各组需求的最大值。每次处理新的一组数据，都要先清空这一组编号范围内的计数。答案可能接近 2×10^{10}，乘法和答案都用 long long。

【参考代码】

```
#include<bits/stdc++.h>
using namespace std;

const int N = 200010;
int t,a,b,k,x[N],y[N],c[N],d[N];

int main(){
  freopen("dance.in","r",stdin);
  freopen("dance.out","w",stdout);
  ios::sync_with_stdio(false);
  cin.tie(0);
  cin >> t;
  while(t--){
    cin >> a >> b >> k;
    for(int i = 1; i <= k; i++) cin >> x[i];
    for(int i = 1; i <= k; i++) cin >> y[i];
    for(int i = 1; i <= a; i++) c[i] = 0;
    for(int i = 1; i <= b; i++) d[i] = 0;
    for(int i = 1; i <= k; i++){
      c[x[i]]++;
      d[y[i]]++;
    }
    long long ans = 1ll * k * (k - 1) / 2;
    for(int i = 1; i <= a; i++) ans -= 1ll * c[i] * (c[i] - 1) / 2;
    for(int i = 1; i <= b; i++) ans -= 1ll * d[i] * (d[i] - 1) / 2;
    cout << ans << '\n';
  }
  return 0;
}
```

【部分分】

条件	分值	做法与复杂度
所有组的 k 之和不超过 2000，第 $1 \sim 6$ 点	30 分	枚举两对的编号 $i < j$，检查 $x_i \neq x_j$ 且 $y_i \neq y_j$，满足就加一。时间 $O(\sum k^2)$，至多约 2000^2 次检查，空间 $O(\max k)$。
每组男生编号两两不同，第 $7 \sim 9$ 点	新增 15 分	任何两对都不会重复使用男生，只需按女生统计冲突，从全部方案中减去同一女生对应的方案数。时间 $O(\sum(a+b+k))$，空间 $O(\max(a+b+k))$。

第二种程序先检查男生编号是否互异；不满足时使用第一种枚举。因此，它覆盖上面两个互不重叠的点组，保证 45 分。在不满足性质的数据组中，这个组合程序的时间为 $O(a+b+k^2)$。

分段求和（job）

【按最后一个位置分类】

从左向右考虑序列。决定是否选择一个新区间时，需要知道前面处理到哪里、已经选了几个区间。固定这两项后，之前的区间具体放在哪里不会再影响后面的选择，只需保留它们能得到的最大总和。

因此，设 $f[j][i]$ 表示在前 i 个位置中，恰好选出 j 个长度为 m 的不重叠区间，能够得到的最大总和。

考虑位置 i 是否被选中：

- 如果不选位置 i，所有区间都在前 $i-1$ 个位置中，最大总和为 $f[j][i-1]$。
- 如果选择位置 i，包含它的区间必须以 i 结尾，所以这个区间是 $[i-m+1,i]$。其余 $j-1$ 个区间必须放在前 $i-m$ 个位置中，它们的最大总和为 $f[j-1][i-m]$。

用前缀和 $s[i]=a_1+\cdots+a_i$ 求出最后一个区间的和，便得到

$$f[j][i]=\max\big(f[j][i-1],\ f[j-1][i-m]+s[i]-s[i-m]\big).$$

两种情况覆盖了所有方案。第二种情况把最后一个区间接在前驱方案之后，位置不会重叠，也不会少选或多选区间，因此两项得到的都是合法方案。

不选任何区间时，总和为 0，所以 $f[0][i]=0$。其他状态先设为负无穷，表示无法达到；特别是 $i<jm$ 时，位置数不足以放下 j 个区间。代码先枚举 j，再从 jm 开始递增枚举 i，这样转移所需的上一行和当前行左边一格都已经算好。答案为 $f[k][n]$。

共有 $O(nk)$ 个状态，每个状态只比较两项，时间复杂度为 $O(nk)$，有效状态需要 $O(nk)$ 空间；参考代码按最大范围预留完整二维数组，约占 191 MiB。答案至多为 $5000\times10^9=5\times10^{12}$，前缀和与状态都要用 long long。

【参考代码】

```
#include<bits/stdc++.h>
using namespace std;

const int N = 5005;
const long long INF = 4000000000000000000LL;
int n,m,k;
long long s[N],f[N][N];

int main(){
  freopen("job.in","r",stdin);
  freopen("job.out","w",stdout);
  cin >> n >> m >> k;
  for(int i = 1; i <= n; i++){
    long long x;
    cin >> x;
    s[i] = s[i - 1] + x;
  }
  for(int j = 1; j <= k; j++){
    for(int i = 0; i <= n; i++) f[j][i] = -INF;
    for(int i = j * m; i <= n; i++){
      f[j][i] = max(f[j][i - 1],f[j - 1][i - m] + s[i] - s[i - m]);
    }
  }
  cout << f[k][n] << '\n';
  return 0;
}
```

【部分分】

下表中的分值是各测试点组的分值，不是算法的累计得分。

条件	分值	做法与复杂度
$n \le 20$	20 分	从左到右搜索每个区间的起点。选完一个区间后，下一段只能从它后面开始；剩余位置不够放下剩余区间时停止。用前缀和计算每段贡献，时间不超过 $O(n2^n)$，空间 $O(n)$。
$m = 1$	10 分	每个区间就是一个位置，排序后取最大的 k 个数。时间 $O(n \log n)$，空间 $O(n)$。
$k = 1$	10 分	搜索只剩一层：枚举唯一一段的起点，用前缀和求区间和并取最大值。时间、空间均为 $O(n)$，复用第一项的 job_1.cpp。
$n \le 500$，$k \le 150$	20 分	令 $h[j][i]$ 表示选了 j 段且最后一段恰好在 i 结束时的最大总和。枚举上一段的右端点 t，取最大的 $h[j-1][t]$，再加上区间 $[i-m+1, i]$ 的和。时间 $O(kn^2)$，空间 $O(kn)$。

搜索同时覆盖测试点 1～4、7～8，保证得到 30 分；排序程序 job_2.cpp 保证通过测试点 5～6，得到 10 分。

最后一档中，上一段的右端点满足 $(j-1)m \le t \le i-m$；选一段时直接计算这段的和，最终在所有合法的最后端点中取最大值。这份程序 job_4.cpp 覆盖测试点 1～4、7～12，保证得到 50 分。进一步发现这些前驱是一个前缀，并复用前缀最大值，就能避免反复枚举 t，得到正文的满分做法。

粉刷木板（paint）

竖刷一块木板时，一笔就能刷满；横刷则可以同时处理多块木板，但会被较矮的木板挡住。因此，可以用一段木板中的最低处，把横刷后剩下的部分分开处理。

考虑区间 $[l,r]$，它们底部高度不超过 h 的区域已经刷好。只记录这段区间和 h，就能确定剩余任务，记最少还需的笔数为 $f(l,r,h)$。

一种做法是每块木板各竖刷一笔，花费 $r-l+1$。

如果有木板不竖刷，它从 h 到这段最低高度 m 的区域就必须全部横刷，至少需要 $m-h$ 笔。这些横刷都可以延长到整个 $[l,r]$：这段内每块木板都至少高 m，重复粉刷也没有影响。因此，可以先用 $m-h$ 笔刷完下面这一整块。

取一个高度为 m 的位置 p。它已经刷完，上方没有木板，后续横刷无法跨过 p。左右两段互不影响，分别从高度 m 继续处理即可。这样既包括全部竖刷，也包括所有需要横刷的最优方案，所以

$$f(l,r,h)=\min\bigl(r-l+1,\ m-h+f(l,p-1,m)+f(p+1,r,m)\bigr).$$

空区间的答案是 0，最终答案为 $f(1,n,0)$。每次扫描当前区间找到最低木板，再递归处理左右两段。

每次非空调用确定一块最低木板，最多有 n 次调用，每次扫描最多 n 块。因此时间复杂度为 $O(n^2)$。数组和最深 n 层的递归一共使用 $O(n)$ 空间。

【参考代码】

```
#include<bits/stdc++.h>
using namespace std;

const int N = 5010;
int n,a[N];

int dfs(int l,int r,int h){
  if(l > r) return 0;
  int p = l;
  for(int i = l + 1; i <= r; i++){
    if(a[i] < a[p]) p = i;
  }
  int ans = a[p] - h;
  ans += dfs(l,p - 1,a[p]);
  ans += dfs(p + 1,r,a[p]);
  return min(r - l + 1,ans);
}

int main(){
  freopen("paint.in","r",stdin);
  freopen("paint.out","w",stdout);
  cin >> n;
  for(int i = 1; i <= n; i++) cin >> a[i];
  cout << dfs(1,n,0) << '\n';
  return 0;
}
```

【部分分】

下表分值是对应点组的分值，三个点组分别计分。

条件	分值	做法与复杂度
$n \leq 18$，第 $1 \sim 4$ 点	20 分	枚举哪些木板竖刷，其他木板全部横刷。每种选择扫描一次，统计竖刷笔数和横刷连续段数，取最小值。时间 $O(n2^n)$，空间 $O(n)$。
所有 $a_i \leq 2$，第 $5 \sim 8$ 点	20 分	最底层横刷一笔，第二层每个高度为 2 的连续段各刷一笔，再和全部竖刷的 n 笔比较。时间 $O(n)$，空间 $O(n)$。
高度非递减，第 $9 \sim 12$ 点	20 分	枚举横刷到高度 a_i。前 i 块可全部横刷完，其余每块竖刷一笔，代价 $a_i + n - i$；再和 n 比较。相同高度枚举最后一个位置即可，程序枚举全部位置也不会漏。时间 $O(n)$，空间 $O(n)$。

高度不超过 2 时，若不是全部竖刷，最底层至少需要一笔横刷；每个高度为 2 的连续段还至少需要一笔横刷或竖刷。因此，表中的两种方案已经达到最少笔数。

枚举竖刷集合时，不能把竖刷过的木板当成空缺：横刷仍可经过它。设相邻两块没有竖刷的木板为 j, i，区间 $[j, i]$ 的最低高度是 b。高度不超过 b 的横刷可以从 j 延伸到 i，更高的部分必须新开 $a_i - b$ 笔。第一块没有竖刷的木板需要 a_i 笔。扫描时维护两块之间的最低高度，就能在线性时间算完一种选择。

非递减这一档中，任何保留给横刷的木板都会要求刷到它的全部高度；设这些木板中的最高高度为 H。由于各高度对应的木板都连成后缀，H 笔横刷足以覆盖所有不高于 H 的木板，剩下的木板竖刷。因此只需枚举 $H = 0$ 或某个 a_i。